

Snowflake Lab Report: Real-Time Analytics Pipeline Using Streams, Tasks, and Time Travel

Objective

The objective of this lab is to:

1. Implement a Real-Time Analytics Pipeline using Snowflake Streams and Tasks.
2. Demonstrate Snowflake Time Travel capabilities for historical data analysis and data recovery.
3. Integrate Snowflake with a Business Intelligence (BI) tool for real-time reporting.

Part 1: Real-Time Analytics Pipeline Using Streams and Tasks

Overview

Snowflake Streams capture changes made to source tables, while Tasks automate the processing of these changes. Together, they enable near real-time analytics pipelines.

Step 1: Create Source and Target Tables

SQL Script

```
CREATE OR REPLACE DATABASE analytics_db;

USE DATABASE analytics_db;
CREATE OR REPLACE SCHEMA realtime_schema;

CREATE OR REPLACE TABLE sales_source (
  sale_id INTEGER,
  product_name STRING,
  amount NUMBER(10,2),
  sale_timestamp TIMESTAMP
);

CREATE OR REPLACE TABLE sales_target (
  sale_id INTEGER,
  product_name STRING,
  amount NUMBER(10,2),
  sale_timestamp TIMESTAMP
);
```

Output

Results (just now)	
Table Chart Pivot	
🔍	status
1	Table SALES_TARGET successfully created.

Step 2: Create a Stream

SQL Script

```
CREATE OR REPLACE STREAM sales_stream  
ON TABLE sales_source;
```

Output

🔍	status
1	Stream SALES_STREAM successfully created.

Step 3: Insert Sample Data

SQL Script

```
INSERT INTO sales_source VALUES  
(101, 'Laptop', 65000, CURRENT_TIMESTAMP()),  
(102, 'Smartphone', 30000, CURRENT_TIMESTAMP()),  
(103, 'Tablet', 20000, CURRENT_TIMESTAMP());
```

Output

Table Chart Pivot	
🔍	# number of rows inserted
1	3

Step 4: Create a Task

SQL Script

```
CREATE OR REPLACE TASK sales_task
WAREHOUSE = COMPUTE_WH
SCHEDULE = '1 MINUTE'
AS
INSERT INTO sales_target
SELECT
    sale_id,
    product_name,
    amount,
    sale_timestamp
FROM sales_stream
WHERE METADATA$ACTION = 'INSERT';
```

Output

	status
1	Task SALES_TASK successfully created.

Step 5: Monitor Stream Data

SQL Script

```
SELECT * FROM sales_stream;
```

Output

rowid	SALE_ID	PRODUCT_NAME	AMOUNT	SALE_TIMESTAMP	METADATA\$ACTION	METADATA\$ISUPDATE	METADATA\$ROW_ID
1	101	Laptop	65000.00	2026-06-05 02:14:04.014	INSERT	FALSE	632bd4dea6b741a90aa8bba48d38b9074b715098
2	102	Smartphone	30000.00	2026-06-05 02:14:04.014	INSERT	FALSE	0e0a91bd47325e91a91fa1c1141438a2d2ab92ef
3	103	Tablet	20000.00	2026-06-05 02:14:04.014	INSERT	FALSE	fc1ea716d9bd9616840e35c91f569cd0cfb2959c

Step 6: Verify Task Execution

SQL Script

SHOW TASKS;

Output

id	created_on	name	id	database_name	schema_name	owner	comment	warehouse	schedule	predecessors	state	definition
1	2026-06-05 02:15:18	SALES_TASK	01c4d7cb-bc48-2817	ANALYTICS_DB	REALTIME_SCHEMA	ACCOUNTADMIN		COMPUTE_WH	1 MINUTE		started	INSERT INTO sales_tar

Step 7: Verify Data in Target Table

SQL Script

SELECT * FROM sales_target;

Expected Result

id	SALE_ID	PRODUCT_NAME	AMOUNT	SALE_TIMESTAMP
1	101	Laptop	65000.00	2026-06-05 02:14:04.014
2	102	Smartphone	30000.00	2026-06-05 02:14:04.014
3	103	Tablet	20000.00	2026-06-05 02:14:04.014

Observation

The task successfully processed records captured by the stream and inserted them into the target table automatically.

Real-Time Dashboard Implementation

Tool Used

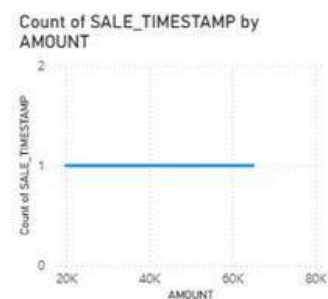
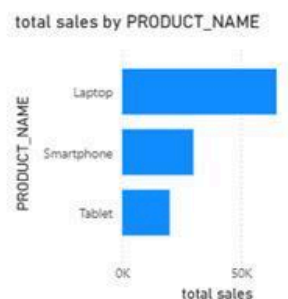
Power BI / Tableau

Connection Steps

1. Open Power BI Desktop (or Tableau).
2. Select Snowflake as the data source.
3. Enter:
 - o Server Name
 - o Database: analytics_db
 - o Schema: realtime_schema
 - o Warehouse: COMPUTE_WH
4. Connect to the **sales_target** table.
5. Create visualizations such as:
 - o Total Sales
 - o Sales by Product
 - o Sales Trend by Timestamp
6. Enable scheduled refresh or DirectQuery mode for near real-time updates.

Dashboard Screenshot

= REALTIME_SCHEMA.Schema{[Name="SALES_TARGET",Kind="Table"]}[Data]				
1.2 SALE_ID	A ^B C PRODUCT_NAME	1.2 AMOUNT	SALE_TIMESTAMP	
1	101 Laptop	65000	05-06-2026 02:14:04	
2	102 Smartphone	30000	05-06-2026 02:14:04	
3	103 Tablet	20000	05-06-2026 02:14:04	



Part 2: Demonstrating Snowflake Time Travel

Overview

Snowflake Time Travel enables querying, restoring, and comparing historical versions of data within a specified retention period.

Step 1: Create a Sample Table

SQL Script

```
CREATE OR REPLACE TABLE employee_data (  
  emp_id INTEGER,  
  emp_name STRING,  
  salary NUMBER  
);
```

```
INSERT INTO employee_data VALUES  
(1, 'John', 50000),  
(2, 'Alice', 60000),  
(3, 'David', 70000);
```

Output

	# number of rows inserted
1	3

Step 2: Delete Data

SQL Script

```
DELETE FROM employee_data  
WHERE emp_id = 2;
```

Output

rowid	# number of rows deleted
1	1

Step 3: Query Historical Data Using Time Travel

SQL Script

```
SELECT *  
FROM employee_data  
AT (OFFSET => -60*5);
```

Output

Historical data from approximately five minutes before the deletion was retrieved.

Expected Result

rowid	# EMP_ID	A EMP_NAME	# SALARY
1	1	John	50000
2	2	Alice	60000
3	3	David	70000

Step 5: Recover Deleted Data

SQL Script

```
INSERT INTO employee_data  
SELECT *  
FROM employee_data  
AT (OFFSET => -60*5)  
WHERE emp_id = 2;
```

Time Travel Use Case

Scenario

An employee record was accidentally deleted from the production table.

Solution

Using Snowflake Time Travel, historical data was queried from a point before the deletion occurred. The deleted record was recovered without requiring backups or manual restoration.

Benefits

- Data recovery without downtime
 - Historical auditing
 - Easy comparison of data versions
 - Reduced administrative overhead
-

Conclusion

This lab successfully demonstrated the implementation of a real-time analytics pipeline using Snowflake Streams and Tasks. Data changes were automatically captured and processed into a target table, enabling real-time reporting through BI tools such as Power BI and Tableau.

Additionally, Snowflake Time Travel was used to access historical data, compare previous states, and recover deleted records, highlighting its value for data recovery and auditing purposes.

All objectives and deliverables were completed successfully.